fundlake: technical note

Governed data platform for a simulated asset manager with an insurance book · v0.1.0 · build 77a376fd6d15dd56

Synthetic data throughout. Names are invented; identifiers carry valid check digits but are not issued. Faults are planted by `fundlake.synth` and recorded in `answer_key.json`.

Architecture



Six sources (17 extracts) land as CSV and are checked against YAML data contracts (schema drift, nulls, types, enums, ISIN/SEDOL/CUSIP check digits, volume). dbt builds bronze (all-varchar, with `_source_file` and `_row_number`), silver (typed, deduplicated, identifiers resolved as of date, findings) and gold (star schema, reconciliation, NAV control, marts). A rate-based gate sits between silver and gold; a failure stops the run with gold untouched. Each build writes an audit record: input hashes, dbt code hash, versions, row counts, gate results and a hash of the gold layer.

Two warehouses, one codebase

Warehouse-specific SQL is confined to eight adapter-dispatched macros in `dbt/macros/dialect.sql`; everything else is ANSI SQL, with joins instead of correlated subqueries. The DuckDB path runs end to end in tests and under Airflow 3.1 (all five tasks succeed; on corrupted data the gate fails and gold is skipped). The PySpark ingestion step, run locally on Spark 3.5.6, produces all 17 bronze tables identical to DuckDB's. Every model renders for Snowflake and parses as Snowflake SQL; `scripts/run_snowflake.py` and `scripts/run_databricks.sh` run the build on those platforms and compare every table, but **have not yet been run on a Snowflake or Databricks account**.

Results (reference build)

Check	Result
Planted faults found / planted; false positives	156 / 156; 0 (19 fault classes, each with recall and precision 1.0)
dbt tests (silver / gold)	silver: 37 models built, 28 tests pass; gold: 27 models, 37 tests pass, 6 warn (the warnings are the planted faults surfacing)
Quality gate	10 of 10 thresholds pass on the reference data; a corrupted identifier file halts the run at the gate (tested)
NAV control	35 of 256 fund-days beyond 0.5 bp, max 51.2 bp; 0 unexplained; clean days agree with the administrator to under £1
Private markets	DPI, RVPI, TVPI and IRR for 10 commitments match an independent Python calculation (IRR within 1e-6); a missing call found by capital-account reconciliation; secondary on purchase price (1.55x) and GP basis (1.34x)
Reproducibility	Two builds from the same inputs give the same build id and gold hash (tested)
Agent evaluation (scripted model)	30 questions: accuracy 100% on 24 answerable, abstention recall 100%, hallucinated numbers 0

Agentic workflow

A PydanticAI agent answers control questions through six evidence tools (`explain_break`, `trace_lineage`, `check_price_staleness`, `summarise_nav_move`, `summarise_private_fund`, `list_findings`), also served over MCP. Output is a typed `Answer`; every number must appear in a tool result (checked), and the agent abstains on unknown funds, dates outside the data, forecasts and questions no tool covers. CI uses a **scripted model that is not an LLM**: it tests tool wiring, schemas, the faithfulness checker and the evaluation harness, and two deliberately faulty variants (one that invents a figure, one that asserts a cause for an unexplained break) are caught. **No real LLM run has been made**; `scripts/run_agent_live.py` does it with an API key.

Not run, and known limits

Not yet run: Snowflake and Databricks accounts (runners written and tested offline); any real-LLM evaluation. Limits: detection rules were written knowing the planted fault classes, so recall on unseen fault types is untested; the DuckDB client view enforces row-level security in Python, mirroring but not identical to Snowflake row access policies; single GBP base currency; no accrued interest or subscriptions.